UNIT II — Data Manipulation using Pandas

Syllabus: Pandas Objects – DataFrames and Series | Data Indexing and Selection (loc, iloc) | Operating on Data (Element-wise and Broadcasting) | Handling Missing Data & Imputation | Hierarchical Indexing and MultiIndex Objects | Combining Datasets (concat, append, merge, join) | Aggregation and Grouping (GroupBy) | Pivot Tables and Cross-tabulation | Data Visualization for EDA (Matplotlib, Seaborn, Plotly)

---

1. Introduction to Data Manipulation with Pandas

1.1 Why Data Manipulation?

- Machine learning models need **clean data** to train and test on.
- Real-world datasets commonly contain:
  - Missing values (`NaN`, `Null`, `NA`)
  - Irrelevant or redundant rows/columns
- **Data Wrangling** is the process of cleaning and reshaping data so it is ready for use.

1.2 Why Pandas?

- Pandas is a Python library built on top of **NumPy**.
- Two core objects:
  - **Series** – a one-dimensional labeled array
  - **DataFrame** – a two-dimensional, multi-dimensional-labeled table built from Series
- Supports heterogeneous data types and missing values.
- Offers operations similar to spreadsheets and SQL.

1.3 Installing Pandas

bash
```
pip install pandas
```

---

2. Creating and Exploring a DataFrame

2.1 Creating a DataFrame

python
```
import pandas as pd

student_register = pd.DataFrame()
student_register['Name'] = ['Abhijit', 'Smriti', 'Akash', 'Roshni']
student_register['Age'] = [20, 19, 20, 14]
student_register['Student'] = [False, True, True, False]

print(student_register)
```

Output

```
    Name  Age  Student
0  Abhijit   20    False
1   Smriti   19     True
2    Akash   20     True
3   Roshni   14    False
```

The index column `[0, 1, 2, 3]` is auto-generated by Pandas and can be customized.

2.2 Adding a Row with `append()`

python
```
new_person = pd.Series(['Mansi', 19, True], index=['Name', 'Age', 'Student'])
student_register = student_register.append(new_person, ignore_index=True)
```

- A DataFrame is essentially a stack of **Series** objects (rows).
- Appending a row = creating a new Series and adding it to the DataFrame.
- *(Note: `.append()` is deprecated in modern Pandas — prefer `pd.concat()`.)*

2.3 Exploring a Dataset

Function	Purpose
`.shape`	Returns dimensions `(rows, columns)`
`.info()`	Column types, non-null counts, memory usage
`.describe()`	Statistical summary (count, mean, std, min, max, quartiles)
`.corr()`	Correlation between numeric columns

python
```
print(student_register.shape)
print(student_register.info())
print(student_register.describe())
```

`.describe()` gives:

- `count` – total entries
- `mean` – average value
- `std` – standard deviation
- `min` / `max` – range
- `25% / 50% / 75%` – quartiles (useful for understanding distribution in ML)

2.4 Dropping Columns and Rows

python
```
students = student_register.drop('Age', axis=1)   # drop column
students = students.drop(2, axis=0)               # drop row
```

---

3. Pandas Objects: Series and DataFrame

3.1 The Series Object

A **Series** is a one-dimensional array with axis labels (an index). It behaves like a hybrid of a NumPy array and a Python dictionary.

python
```
data = pd.Series([0.25, 0.5, 0.75, 1.0])
```

```
data[1]       # 0.5
data[1:3]     # slice
data.values   # underlying NumPy array
data.index    # the Index object
```

Custom index:

python
```
data = pd.Series([0.25, 0.5, 0.75, 1.0], index=['a', 'b', 'c', 'd'])
data['b']     # 0.5
```

Series as a dictionary:

python
```
population_dict = {'California': 38332521, 'Texas': 26448193, 'New York': 19651127}
population = pd.Series(population_dict)
population['California']          # 38332521
population['California':'Illinois'] # slicing by label
```

Ways to construct a Series:

python
```
pd.Series([2, 4, 6])               # from a list
pd.Series(5, index=[100, 200, 300])  # from a scalar
pd.Series({2: 'a', 1: 'b', 3: 'c'})  # from a dictionary
```

3.2 The DataFrame Object

A **DataFrame** is a collection of aligned Series — a 2D, table-like structure with row and column indices.

python
```
area = pd.Series({'California': 423967, 'Texas': 695662, 'New York': 141297})
states = pd.DataFrame({'population': population, 'area': area})

states.index     # row labels
states.columns   # column labels
states['area']   # access a column (returns a Series)
```

Ways to construct a DataFrame:

python
```
pd.DataFrame(population, columns=['population'])        # from a Series
pd.DataFrame([{'a': i, 'b': 2*i} for i in range(3)])     # from a list of dicts
pd.DataFrame({'population': population, 'area': area})   # from a dict of Series
pd.DataFrame(np.random.rand(3, 2), columns=['foo', 'bar'], index=['a', 'b', 'c'])  # from NumPy array
```

3.3 The Index Object

`Index` objects are **immutable** arrays used to label rows/columns and support set operations.

python
```
ind = pd.Index([2, 3, 5, 7, 11])
ind[1]     # 3
ind[::2]   # [2, 5, 11]
ind.size, ind.shape, ind.ndim, ind.dtype

indA = pd.Index([1, 3, 5, 7, 9])
indB = pd.Index([2, 3, 5, 7, 11])
```

```
indA & indB   # intersection → [3, 5, 7]
indA | indB   # union → [1, 2, 3, 5, 7, 9, 11]
indA ^ indB   # symmetric difference → [1, 2, 9, 11]
```

---

4. Data Indexing and Selection

Pandas indexing extends NumPy-style indexing (`arr[2,1]`, slicing, masking, fancy indexing) with label-aware features.

4.1 Selection in a Series

As a dictionary:

python
```
data = pd.Series([0.25, 0.5, 0.75, 1.0], index=['a', 'b', 'c', 'd'])
data['b']            # 0.5
'a' in data          # True
data.keys()
list(data.items())
data['e'] = 1.25       # Series are mutable, like dicts
```

As a one-dimensional array:

python
```
data['a':'c']                  # explicit slice — 'c' is INCLUDED
data[0:2]                      # implicit slice — index 2 EXCLUDED
data[(data > 0.3) & (data < 0.8)]   # masking
data[['a', 'e']]               # fancy indexing
```

4.2 `loc`, `iloc` (and the deprecated `ix`)

When a Series has an **integer index**, `data[1]` and `data[1:3]` can be ambiguous — is it label-based or position-based?

python
```
data = pd.Series(['a', 'b', 'c'], index=[1, 3, 5])

data.loc[1]     # explicit/label index → 'a'
data.loc[1:3]   # label-based, inclusive of both ends

data.iloc[1]    # implicit/position index → 'b'
data.iloc[1:3]  # position-based, exclusive of end
```

Indexer	Basis	Notes
`loc`	Explicit (label) index	Inclusive of end in slices
`iloc`	Implicit (position) index	Exclusive of end, like Python lists
`ix`	Hybrid of both	**Deprecated** — avoid

Best practice: always prefer `loc`/`iloc` over plain `[]` for clarity and to avoid bugs.

4.3 Selection in a DataFrame

A DataFrame behaves both as a **dictionary of Series** and as a **2D array**.

As a dictionary of columns:

python
```
data['area']    # dictionary-style
data.area       # attribute-style (fails if column name isn't a valid identifier or clashes with a method, e.g. 'pop')
data['density'] = data['pop'] / data['area']   # add a new column
```

As a 2D array:

python
```
data.values         # raw NumPy array
data.T              # transpose

data.iloc[:3, :2]                    # implicit indexing
data.loc[:'Illinois', :'pop']          # explicit indexing
data.loc[data.density > 100, ['pop', 'density']]   # boolean mask + fancy indexing
data.iloc[0, 2] = 90                   # setting a value
```

Row-focused shortcuts:

python
```
data['Florida':'Illinois']   # slice by row label
data[1:3]                 # slice by row position
data[data.density > 100]     # row-wise boolean mask
```

---

5. Operating on Data: Element-wise Operations & Broadcasting

Index Alignment

When performing binary operations, Pandas **aligns indices automatically**; unmatched labels produce `NaN`.

python
```
area = pd.Series({'Alaska': 1723337, 'Texas': 695662, 'California': 423967})
pop  = pd.Series({'California': 38332521, 'Texas': 26448193, 'New York': 19651127})
print(pop / area)                  # NaN where labels don't match

pop.add(area, fill_value=0)        # control the fill value instead of NaN
```

This also applies to DataFrames, aligning **both rows and columns**.

Output:

```
Alaska         NaN
California    90.413
Texas         38.019
New York       NaN
dtype: float64
```

Operator ↔ Method mapping:

Operator	Method
`+`	`add()`
`-`	`sub()`, `subtract()`

Operator	Method
*	mul(), multiply()
/	truediv(), divide()
//	floordiv()
%	mod()
**	pow()

Element-wise Operations vs. Broadcasting

- **Element-wise:** both arrays must be the **same shape**; operation applied position by position.

Example :

```
a = np.array([1, 2, 3]);
b = np.array([4, 5, 6])
a + b   # [5, 7, 9]
```

- **Broadcasting:** the smaller array is automatically "stretched" to match the larger array's shape.

Example 1:

```
a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([10, 20, 30])
a + b   # [[11, 22, 33], [14, 25, 36]]
```

Example 2:

```
A = np.array([1, 2, 3])
B = 2

print(A * B)
```

Output:

```
[2 4 6]
```

---

6. Handling Missing Data

6.1 What Counts as Missing Data?

- Occurs when no value is recorded for an item.
- Pandas refers to missing data generically as **NA (Not Available)**.
- Represented by:
  - **`None`** — Python's singleton object; only usable in `object`-dtype arrays.
  - **`NaN`** — "Not a Number," an IEEE floating-point value; preferred for numeric data because it supports fast vectorized computation.

6.2 Detecting Missing Values

Functions:

```
df.isna()       # Returns True where data is missing
df.isnull()     # Same as isna()
df.notnull()    # Returns True where data is present
```

```
df[df.isnull().any(axis=1)]
```

→Displays rows with at least one missing value

```
df.isnull().sum()
```

→Counts missing values column-wise

6.3 Handling Missing Values

Function	Purpose
`isnull()`	Detect missing values
`notnull()`	Opposite of `isnull()`
`dropna()`	Remove rows/columns containing missing data
`fillna(value)`	Replace `NaN` with a specified value
`replace()`	Replace specific values (not limited to NaN) using a value, regex, list, or dict
`interpolate()`	Estimate and fill missing values using interpolation

---

7. Hierarchical Indexing (MultiIndex)

7.1 What Is a MultiIndex?

- A **hierarchical index** allows more than one level of row/column labeling — useful when data has more than two logical dimensions.
- Structure: Series (1D) → DataFrame (2D) → MultiIndex DataFrame (3D+).

7.2 Creating a MultiIndex

Example:

```
index = pd.MultiIndex.from_tuples( [('2023', 'Jan'), ('2023', 'Feb'), ('2024', 'Jan')], names=['Year', 'Month'])
df = pd.DataFrame({'Sales': [1000, 1200, 1300], 'Profit': [200, 250, 300]}, index=index)
```

Output:

		Sales	Profit
Year	Month		
2023	Jan	1000	200
	Feb	1200	250
2024	Jan	1300	300

7.3 Selecting from a MultiIndex

```
df.loc['2023']          # all rows for year 2023
df.loc[('2023', 'Jan')]   # a specific row
```

7.4 Reorganizing a MultiIndex

Operation	Purpose
`swaplevel(0, 1)`	Swap the position of two index levels
`sort_index(level=1)`	Sort based on a given index level
`.sum(level='exam')`	Aggregate statistics per index level
`set_index(['a', 'b'])`	Move columns into the row index
`reset_index()`	Move index levels back into columns

7.5 MultiIndex in Columns

```
arrays = [['Sales', 'Sales', 'Profit', 'Profit'], ['Q1', 'Q2', 'Q1', 'Q2']]
column_index = pd.MultiIndex.from_arrays(arrays, names=['Metric', 'Quarter'])
df = pd.DataFrame([[100, 150, 20, 25], [200, 250, 30, 35]], columns=column_index)
```

Output:

```
Metric      Sales      Profit
Quarter    Q1   Q2     Q1  Q2
0         100  150     20  25
1         200  250     30  35
```

Reshaping operations:

- `stack()` — converts a column level into an index level
- `unstack()` — converts an index level into a column level
- `reset_index()` — flattens a MultiIndex

---

8. Combining Datasets

8.1 `concat()` — Stacking DataFrames

Usage	Description
`pd.concat([df1, df2])`	Vertical stacking (adds rows)
`pd.concat([df1, df2], axis=1)`	Horizontal stacking (adds columns)
Different indexes	Aligns by index; missing values filled with `NaN`
`pd.concat([df1, df2], keys=['df1','df2'])`	Adds a hierarchical (MultiIndex) grouping

python

```
pd.concat([df1, df2])                  # vertical
pd.concat([df1, df2], axis=1)          # horizontal
pd.concat([df3, df4], axis=1)          # different indexes → NaN for gaps
pd.concat([df1, df2], keys=['df1', 'df2'])   # MultiIndex result
```

8.2 `append()` — Adding Rows

Returns a **new** DataFrame; the originals are unchanged unless reassigned. *(Deprecated in modern Pandas in favor of `pd.concat()`.)*

python
```
df1.append(df2, ignore_index=True)     # simple row append
df3.append(df4, ignore_index=True)     # mismatched columns → NaN filled in automatically
df5.append(pd.Series({'A': 5, 'B': 6}), ignore_index=True)   # append a single row (Series)
```

8.3 `merge()` — Combining by Column Values (SQL-style)

python
```
pd.merge(df1, df2, on='key', how='inner')   # only matching keys
pd.merge(df1, df2, on='key', how='left')    # all from left, matched from right
pd.merge(df1, df2, on='key', how='outer')   # all rows from both, NaN where unmatched
```

8.4 `join()` — Combining by Index

python
```
df3.join(df4, how='inner')   # only matching index labels
df3.join(df4, how='left')    # all rows from df3, matched from df4
```

8.5 Summary: merge vs. join

Basis	`how='inner'`	`how='left'`	`how='outer'`
`merge()`	Column(s)	Matching keys only	All from left	All from both
`join()`	Index	Matching index only	All from left	All from both

---

9. Aggregation and Grouping

9.1 `groupby()` Basics

python
```
df = pd.DataFrame({'Category': ['A','A','B','B','C'], 'Value': [10,15,10,20,25]})
grouped = df.groupby('Category')

grouped.sum()                    # aggregate with sum
grouped.agg(['sum', 'mean', 'count'])  # multiple aggregations at once

def range_func(x):
    return x.max() - x.min()
grouped.agg(range_func)              # custom aggregation function
```

9.2 Grouping by Multiple Columns

python
```
df_multi = pd.DataFrame({
    'Category': ['A','A','B','B','C'],
    'SubCategory': ['X','Y','X','Y','X'],
    'Value': [10,15,10,20,25]
})
df_multi.groupby(['Category', 'SubCategory']).sum()
```

---

10. Pivot Tables and Cross-tabulation

10.1 Creating a Pivot Table

python

```
pd.pivot_table(df_multi, values='Value', index=['Category'], columns=['SubCategory'], aggfunc='sum')
```

10.2 Full Syntax

python
```
pd.pivot_table(
    data, values=None, index=None, columns=None,
    aggfunc='mean', fill_value=None, margins=False,
    dropna=True, margins_name='All', observed=False
)
```

10.3 Key Parameters

Parameter	Description
`data`	The DataFrame to pivot
`values`	Column(s) to aggregate
`index`	Row grouping labels
`columns`	Column grouping labels
`aggfunc`	Aggregation function(s): string, NumPy function, or dict
`fill_value`	Value used to replace missing entries
`margins`	Adds subtotal row/column when `True`
`margins_name`	Custom label for the totals row/column
`dropna`	Whether to drop groups that are entirely NaN
`observed`	How categorical data is handled

10.4 Common Techniques

python
```
pd.pivot_table(data, values='Sales', index='Product', columns='Region', fill_value=0)          # fill missing values
pd.pivot_table(data, values='Sales', index='Product', columns='Region', margins=True, margins_name='Total')
# add totals
pd.pivot_table(data, values='Sales', index='Region', columns='Product', aggfunc=['sum','mean','count'])      #
multiple aggfuncs
pd.pivot_table(data, values='Sales', index=['Region','Product'], columns='Quarter', aggfunc=['sum','count'])  #
multi-level index
```

Custom aggregation functions can also be passed directly to `aggfunc`.

10.5 Pivot Table vs. GroupBy

Feature	Pivot Table	GroupBy
Object returned	DataFrame	DataFrameGroupBy (intermediate)
Orientation	Two-dimensional (rows & columns)	One-dimensional
Flexibility	Works like an Excel pivot	More flexible in code pipelines
Method	`pivot_table()`	`groupby()` + aggregation

10.6 Filtering a Pivot Table

python
```
pivot_df[pivot_df['Sales'] > 5000]                  # constant threshold
pivot_df[pivot_df['Sales'] > pivot_df['Sales'].mean()] # dynamic threshold
```

---

11. Vectorized String Operations

The `.str` accessor applies Python string methods **element-wise** across an entire Series, automatically skipping missing values — no explicit loops required.

11.1 Common `.str` Methods

Operation	Method
Lowercase	`str.lower()`
Uppercase	`str.upper()`
String length	`str.len()`
Check substring	`str.contains()`
Replace substring	`str.replace()`
Split / join	`str.split()`, `str.join()`
Remove whitespace	`str.strip()`
Extract substring	`str[:N]`
Concatenate strings	`+` operator
Count / find	`str.count()`, `str.find()`, `str.findall()`
Start/end check	`str.startswith()`, `str.endswith()`
Case check	`str.islower()`, `str.isupper()`, `str.isnumeric()`
Swap case	`str.swapcase()`
Repeat	`str.repeat(n)`

11.2 Examples

python
```
df = pd.DataFrame({'Text': ['Hello', 'World', 'Pandas']})

df['Text_Lower']    = df['Text'].str.lower()
df['Text_Upper']    = df['Text'].str.upper()
df['Text_Length']   = df['Text'].str.len()
df['Contains_o']    = df['Text'].str.contains('o')
df['Text_Replaced'] = df['Text'].str.replace('o', 'O')
df['Text_Split']    = df['Text'].str.split()
df['Text_Substr']   = df['Text'].str[:3]

df_ws = pd.DataFrame({'Text': [' Hello ', ' World ']})
df_ws['Text_Strip'] = df_ws['Text'].str.strip()

df['Text_Concat'] = df['Text'] + ' ' + df['More_Text']
```

---

12. Data Visualization for EDA

Pandas integrates with visualization libraries to support exploratory data analysis (EDA):

- **Pandas `.plot()`** — quick plots built on Matplotlib; by default plots index vs. every numeric column (specific columns can be selected).
- **Matplotlib** — the foundational plotting library; gives fine-grained control over figures and axes.
- **Seaborn** — built on Matplotlib; simplifies statistical visualizations (distributions, categorical plots, heatmaps).

- **Plotly** — enables interactive, web-based visualizations (zoom, hover tooltips, dynamic charts).

python
```
df.plot()  # quick line plot of all numeric columns
```

---

13. Quick Recap

- **Pandas** builds on NumPy to provide labeled, flexible **Series** and **DataFrame** structures.
- **`loc`/`iloc`** give unambiguous label-based vs. position-based indexing.
- **Broadcasting** and **index alignment** let Pandas operate cleanly on differently-shaped or differently-labeled data.
- **Missing data** (`NaN`/`None`) can be detected (`isnull`), removed (`dropna`), or filled (`fillna`, `interpolate`).
- **MultiIndex** enables higher-dimensional, hierarchical data representation.
- **`concat`, `append`, `merge`, `join`** each combine datasets in different ways — by axis, by column key, or by index.
- **`groupby`** and **`pivot_table`** are the two main tools for aggregation and summarization.
- **`.str`** accessor enables efficient, vectorized text cleaning.
- **Matplotlib, Seaborn, and Plotly** extend Pandas for visual exploratory data analysis.